



CATALYST OPTIMIZER

The Query Planner That Rewrites Your Code Before It Ever Runs

Logical Plan

Rule-Based

Cost-Based

CodeGen

Interview Ready in 6 Slides

Swipe to start learning →

C CATALYST'S FOUR-PHASE PIPELINE

Every query — SQL or DataFrame — passes through the **same four phases** before a single Task is ever scheduled.

Unresolved Code → Executable Plan

1

Analysis — resolves columns, types, and tables against the catalog



2

Logical Optimization — rule-based rewrites: pushdown, pruning, folding



3

Physical Planning — multiple candidate plans scored by a cost model



4

Code Generation — the chosen plan compiles straight to JVM bytecode

Interview Tip: All four phases run on the Driver, before any data moves — Catalyst never touches a row to do its job.

R LOGICAL OPTIMIZATION RULES

APPLIED REGARDLESS OF DATA SIZE

- Predicate Pushdown
- Column Pruning
- Constant Folding
- Boolean Simplification

What Each Rule Actually Does

- 1

Predicate Pushdown — filters move as close to the data source as possible
- ↓
- 2

Column Pruning — only requested columns are read from Parquet/ORC
- ↓
- 3

Constant Folding — expressions like $2 * 60$ computed once, not per row
- ↓
- 4

Boolean Simplification — redundant conditions collapse away entirely

Interview Tip: These rules fire purely on the plan's structure — they run identically whether the table has 10 rows or 10 billion.

P CHOOSING A PHYSICAL PLAN

One Logical Plan can map to **several Physical Plans** — Catalyst's cost model picks the cheapest one using table statistics.

Same Join, Different Strategy

1

A single join can become **Broadcast Hash**, **Sort-Merge**, or **Shuffle Hash** Join



2

Catalyst estimates table sizes from stats to score each candidate



3

Small table (below `autoBroadcastJoinThreshold`) → Broadcast Join, zero shuffle



4

Large-large join → Sort-Merge Join, both sides shuffled and sorted

Interview Tip: Stale table statistics are the #1 reason Catalyst picks the wrong join strategy in production.

G CODEGEN — THE FINAL STEP

Instead of interpreting each operator row by row, Catalyst **fuses a whole chain** of operators into one generated Java function that runs directly on the JVM.

Interpreted vs Generated

```
// Without CodeGen: virtual calls per row, per operator  
filterIter.next() -> projectIter.next() -> aggIter.next()
```

```
// With Whole-Stage CodeGen: one fused loop  
for (row : partition) { if (cond(row)) sum += row.amt; }
```

Interview Tip: This is why DataFrame/SQL code often beats hand-written RDD loops — CodeGen erases the per-row virtual function call overhead.

Save this slide before your next Data Engineering interview

PHASE	WHAT HAPPENS	KEY INTERVIEW FACT
Analysis	Resolves plan	Unresolved → Resolved Logical Plan
Logical Optimization	Rule-based rewrites	Pushdown, pruning, folding
Physical Planning	Cost-based selection	Picks Broadcast vs Sort-Merge Join
Code Generation	Compiles to JVM bytecode	Removes row-by-row overhead

Follow for daily Data Engineering content

@chintapradeep

Spark · Python · SQL · Interview Prep